2. Microservices with API gateway

Activity 1

Question

Go to Airtel website and find out all products and services offered by Airtel.

Answer

Products and Services offered by Airtel:

- Mobile Prepaid
- Mobile Postpaid
- International Roaming
- DTH (Digital TV)
- Airtel Xstream Fiber Broadband
- Wi-Fi Services
- Landline Services
- Airtel Payments Bank
- FASTag
- Airtel Black
- Business Solutions
- Cloud Services
- IoT Solutions
- Data Center Services
- Cyber Security Services
- Enterprise Connectivity
- Airtel Thanks Rewards
- OTT Subscriptions (Xstream Play)

Activity 2

Question

Discuss and come up with ideas to handle the situation where one service becomes slow and affects the whole banking application.

Answer

Possible Solutions:

1. **Use Microservices Architecture**

   Split the application into smaller independent services such as Account Service, Loan Service, Insurance Service, etc.

2. **Load Balancing**

   Distribute requests across multiple instances of the same service.
3. **Auto Scaling**

   Automatically increase service instances during high traffic.
4. **Fault Isolation**

   Failure in one service should not affect other services.
5. **Monitoring and Logging**

   Use monitoring tools to detect memory leaks and performance issues early.
6. **Circuit Breaker Pattern**

   Prevent cascading failures when a service becomes unavailable.
7. **Caching**

   Cache frequently accessed data such as account balances.
8. **API Gateway**

   Route and manage requests efficiently through a centralized gateway.
9. **Containerization**

   Deploy services independently using Docker/Kubernetes.
10. **Regular Performance Testing**

    Identify bottlenecks before deployment.

Activity 3

Question

Creating Microservices for account and loan

Answer

Eureka Project Codes:

pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
	xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
	<modelVersion>4.0.0</modelVersion>
	<parent>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-parent</artifactId>
		<version>4.1.0</version>
		<relativePath/> <!-- lookup parent from repository -->
	</parent>
	<groupId>com.cognizant</groupId>
	<artifactId>eureka</artifactId>
	<version>0.0.1-SNAPSHOT</version>
	<name/>
	<description/>
	<url/>
```

```
	<licenses>
		<license/>
	</licenses>
	<developers>
		<developer/>
	</developers>
	<scm>
		<connection/>
		<developerConnection/>
		<tag/>
		<url/>
	</scm>
	<properties>
		<java.version>17</java.version>
		<spring-cloud.version>2025.1.2</spring-cloud.version>
	</properties>
	<dependencies>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter-webmvc</artifactId>
		</dependency>
		<dependency>
			<groupId>org.springframework.cloud</groupId>
			<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
		</dependency>

		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-devtools</artifactId>
			<scope>runtime</scope>
			<optional>true</optional>
		</dependency>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter-webmvc-test</artifactId>
			<scope>test</scope>
		</dependency>
	</dependencies>
	<dependencyManagement>
		<dependencies>
			<dependency>
				<groupId>org.springframework.cloud</groupId>
				<artifactId>spring-cloud-dependencies</artifactId>
				<version>${spring-cloud.version}</version>
				<type>pom</type>
				<scope>import</scope>
			</dependency>
		</dependencies>
	</dependencyManagement>

	<build>
		<plugins>
			<plugin>
				<groupId>org.springframework.boot</groupId>
				<artifactId>spring-boot-maven-plugin</artifactId>
			</plugin>
		</plugins>
	</build>

</project>
```

EurekaApplication.java:

```
package com.cognizant.eureka;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication

@EnableEurekaServer

public class EurekaApplication {

    public static void main(String[] args) {

        SpringApplication.run(EurekaApplication.class, args);

    }

}
```

application.properties:

```
spring.application.name=eureka
server.port=8761

eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

Loan Project Codes:

pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <parent>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-parent</artifactId>
            <version>3.5.3</version>
      <relativePath/>
      </parent>
    <groupId>com.cognizant</groupId>
    <artifactId>loan</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>loan</name>
    <description>Loan Microservice</description>

    <properties>
        <java.version>17</java.version>
```

```
        <spring-cloud.version>2025.0.0</spring-cloud.version>
    </properties>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>${spring-cloud.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <dependencies>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

LoanController.java:

```
package com.cognizant.loan.controller;

import org.springframework.web.bind.annotation.*;

@RestController
public class LoanController {

    @GetMapping("/loans/{number}")
    public Loan getLoan(@PathVariable String number) {
        return new Loan(number, "car", 400000, 3258, 18);
```

```
    }
}
```

Loan.java:

```
package com.cognizant.loan.controller;

public class Loan {

    private String number;
    private String type;
    private double loan;
    private double emi;
    private int tenure;

    public Loan(String number, String type,
                double loan, double emi, int tenure) {
        this.number = number;
        this.type = type;
        this.loan = loan;
        this.emi = emi;
        this.tenure = tenure;
    }

    public String getNumber() {
        return number;
    }

    public String getType() {
        return type;
    }

    public double getLoan() {
        return loan;
    }

    public double getEmi() {
        return emi;
    }

    public int getTenure() {
        return tenure;
    }
}
```

LoanApplication.java:

```
package com.cognizant.loan;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;

@SpringBootApplication
@EnableDiscoveryClient
public class LoanApplication {

    public static void main(String[] args) {
        SpringApplication.run(LoanApplication.class, args);
    }
}
```

application.properties:

```
spring.application.name=loan
server.port=8081
spring.application.name=loan-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Account Project Codes:

pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <parent>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-parent</artifactId>
            <version>3.5.3</version>
      <relativePath/>
      </parent>

    <groupId>com.cognizant</groupId>
    <artifactId>account</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>account</name>
    <description>Account Microservice</description>

    <properties>
        <java.version>17</java.version>
        <spring-cloud.version>2025.0.0</spring-cloud.version>
    </properties>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>${spring-cloud.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <dependencies>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
```

```
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

Account.java:

```
package com.cognizant.account.controller;

public class Account {

    private String number;
    private String type;
    private double balance;

    public Account(String number, String type, double balance) {
        this.number = number;
        this.type = type;
        this.balance = balance;
    }

    public String getNumber() {
        return number;
    }

    public String getType() {
        return type;
    }

    public double getBalance() {
        return balance;
    }
}
```

AccountController.java:

```
package com.cognizant.account.controller;

import java.util.HashMap;
import java.util.Map;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
```

```java
@RestController
public class AccountController {

    @GetMapping("/accounts/{number}")
    public Map<String,Object> getAccount(@PathVariable String number){

        Map<String,Object> account = new HashMap<>();

        account.put("number", number);
        account.put("type", "savings");
        account.put("balance", 234343);

        return account;
    }
}
```

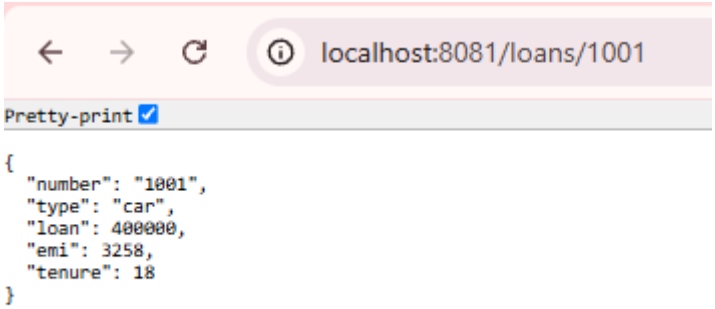
AccountApplication.java:

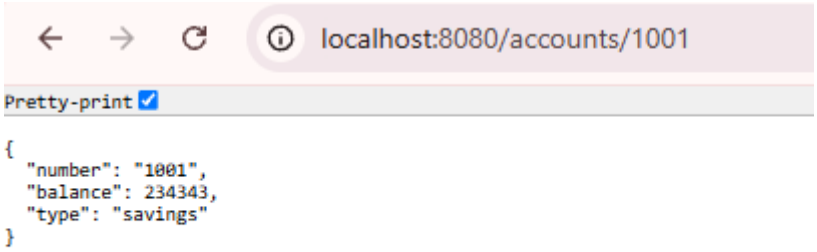
```java
package com.cognizant.account;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;

@SpringBootApplication
@EnableDiscoveryClient
public class AccountApplication {

    public static void main(String[] args) {
        SpringApplication.run(AccountApplication.class, args);
    }
}
```

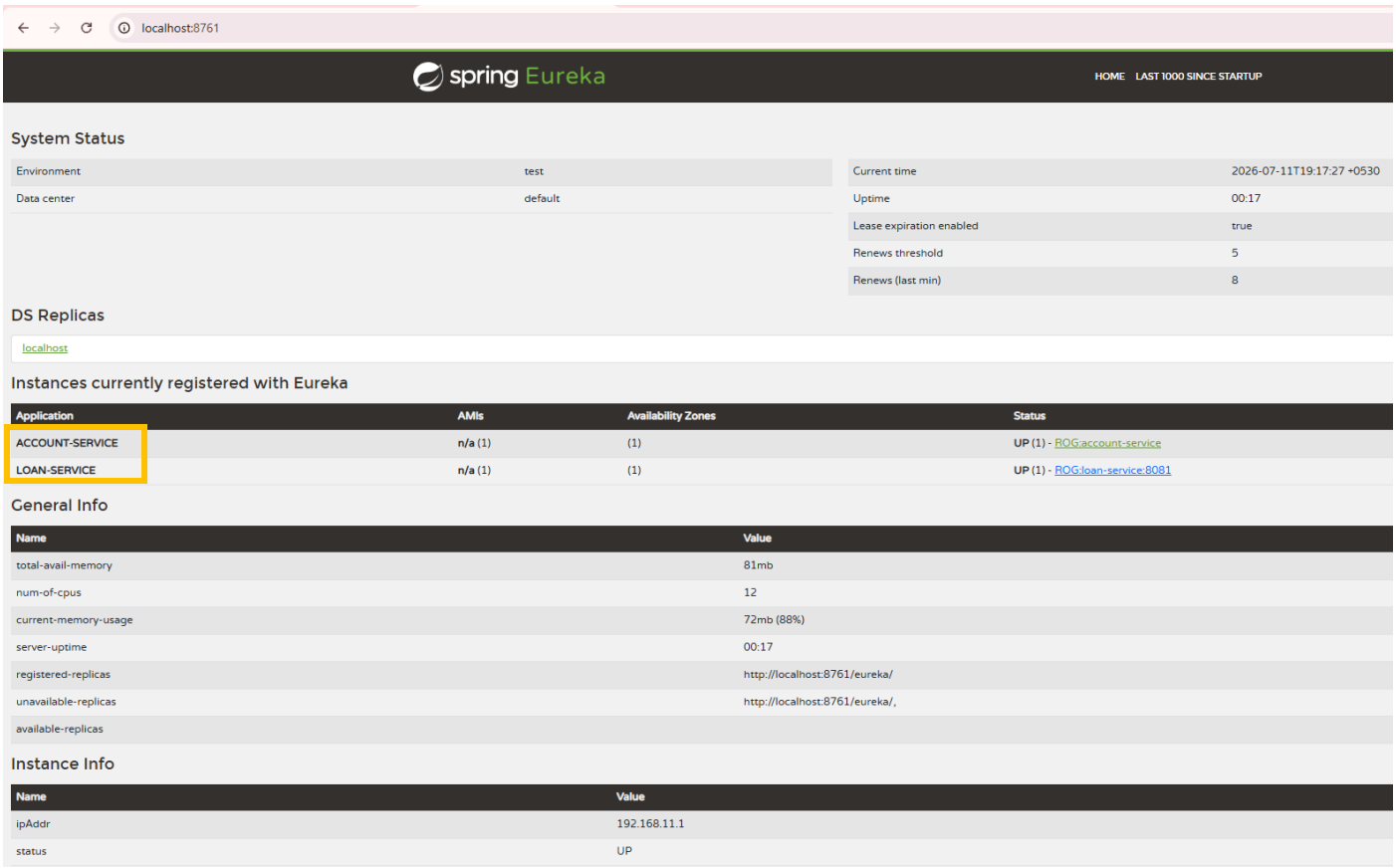
application.properties:

```
spring.application.name=account
spring.application.name=account-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

OUTPUT:




localhost:8080/accounts/1001
Pretty-print
{
"number": "1001",
"balance": 234343,
"type": "savings"
}


localhost:8761
spring Eureka
HOME LAST 1000 SINCE STARTUP
System Status
Environment test
Data center default
Current time 2026-07-11T19:17:27 +0530
Uptime 00:17
Lease expiration enabled true
Renews threshold 5
Renews (last min) 8
DS Replicas
localhost
Instances currently registered with Eureka
Application AMIs Availability Zones Status
ACCOUNT-SERVICE n/a (1) (1) UP (1) - ROG:account-service
LOAN-SERVICE n/a (1) (1) UP (1) - ROG:loan-service:8081
General Info
Name Value
total-avail-memory 81mb
num-of-cpus 12
current-memory-usage 72mb (88%)
server-uptime 00:17
registered-replicas http://localhost:8761/eureka/
unavailable-replicas http://localhost:8761/eureka/,
available-replicas
Instance Info
Name Value
ipAddr 192.168.11.1
status UP